

Neural networks and deep learning(PE - V)(18CSE23)

UNIT - 4

Convolutional neural networks:

The convolution operation

Convolutional networks, also known as convolutional neural networks, or CNNs, are a specialized kind of neural network for processing data that has a known grid-like topology. Examples include time-series data, which can be thought of as a 1-D grid taking samples at regular time intervals, and image data, which can be thought of as a 2-D grid of pixels. Convolutional networks have been tremendously successful in practical applications. The name “convolutional neural network” indicates that the network employs a mathematical operation called convolution. Convolution is a specialized kind of linear operation. Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.

The Convolution Operation

$$s(t) = \int x(a)w(t-a)da.$$

This operation is called convolution. The convolution operation is typically denoted with an asterisk:

$$s(t) = (x * w)(t).$$

In convolutional network terminology, the first argument x to the convolution is often referred to as the input and the second argument w as the kernel. The output is sometimes referred to as the feature map.

The time index t can then take on only integer values. If we now assume that x and w are defined only on integer t , we can define the discrete convolution:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a).$$

Finally, we often use convolutions over more than one axis at a time. For example, if we use a two-dimensional image I as our input, we probably also want to use a two-dimensional kernel K :

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i-m, j-n).$$

Convolution is commutative, meaning we can equivalently write

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n).$$

The commutative property of convolution arises because we have flipped the kernel relative to the input, in the sense that as m increases, the index into the input increases, but the index into the kernel decreases. The only reason to flip the kernel is to obtain the commutative property. While the commutative property is useful for writing proofs, it is not usually an important property of neural network implementation. Instead, many neural network libraries implement a related function called the cross-correlation, which is the same as convolution but without flipping the kernel

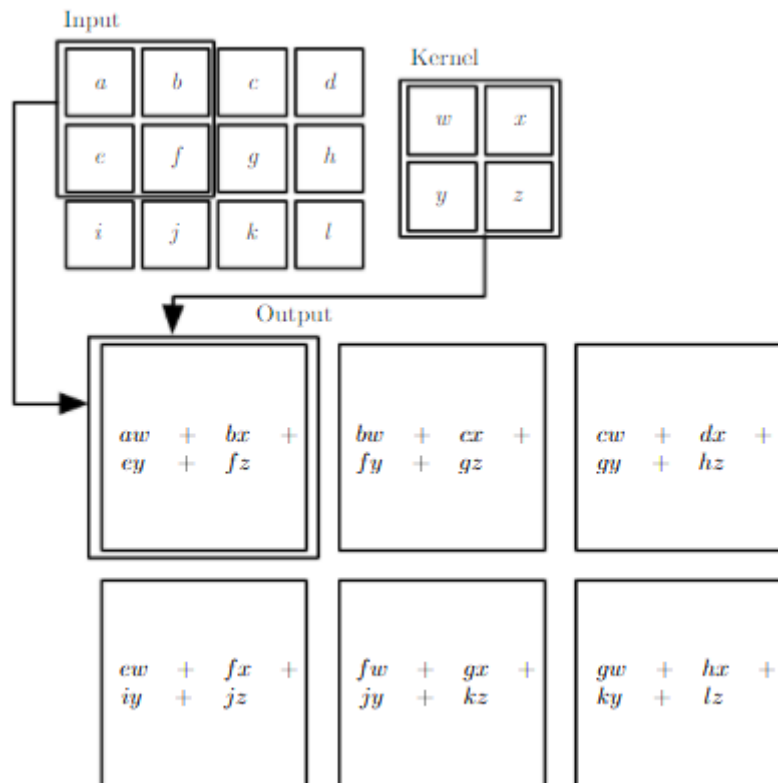
$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Discrete convolution can be viewed as multiplication by a matrix, but the matrix has several entries constrained to be equal to other entries. For example, for univariate discrete convolution, each row of the matrix is constrained to be equal to the row above shifted by one element. This is known as a Toeplitz matrix. In two dimensions, a double block circulant matrix corresponds to convolution.

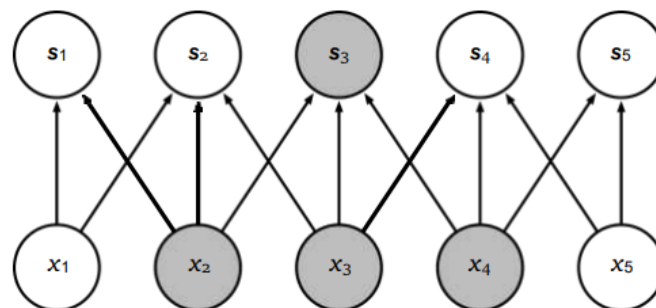
Motivation

Convolution leverages three important ideas that can help improve a machine learning system: sparse interactions, parameter sharing, and equivariant representations. Moreover, convolution provides a means for working with inputs of variable size. Traditional neural network layers use matrix multiplication by a matrix of parameters with a separate parameter describing the interaction between each input unit and each output unit. This means that every output unit interacts with every input unit. Convolutional networks, however, typically have sparse interactions. This is accomplished by making the kernel smaller than the input. For example, when processing an image, the input image might have thousands or millions of pixels, but we can detect small, meaningful features such as edges with kernels that occupy only tens or hundreds of pixels. This means that we need to store fewer parameters, which both reduces the memory requirements of the model and improves its statistical efficiency. It also means that computing the output requires fewer operations. These improvements in efficiency are usually quite large.

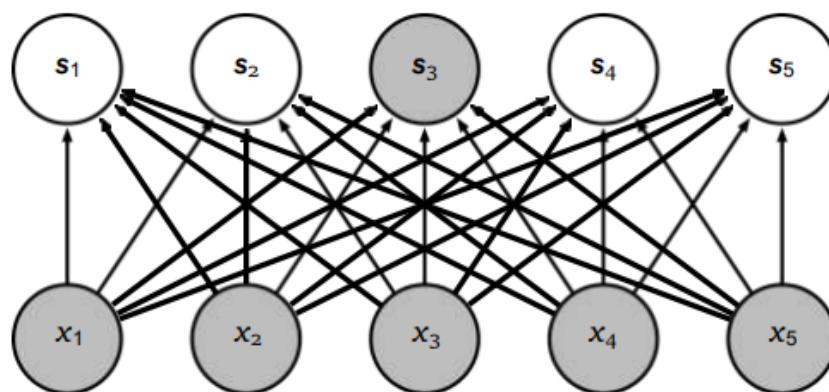
Parameter sharing refers to using the same parameter for more than one function in a model.



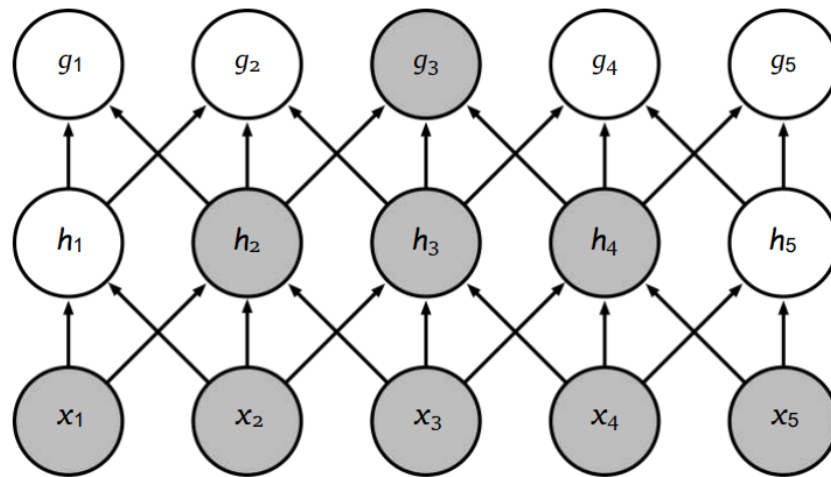
An example of 2-D convolution without kernel flipping.



Sparse connections due to small convolution kernel. Convolution shares the same parameters across all spatial locations



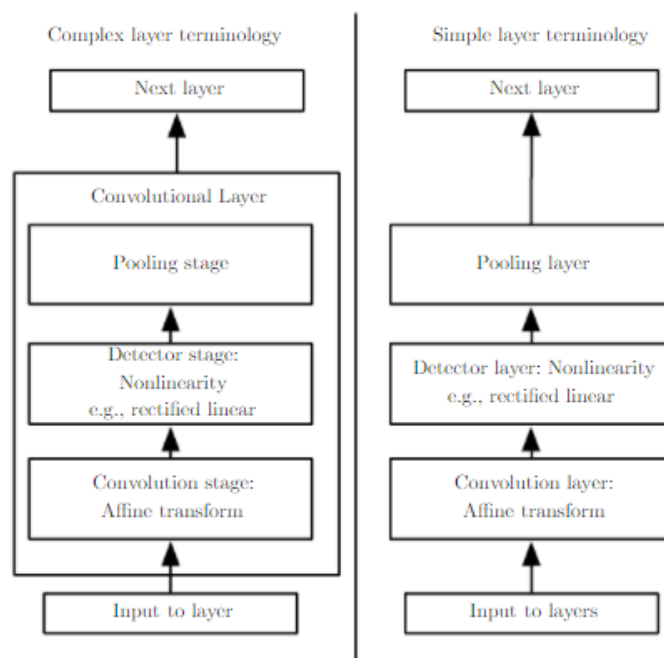
Dense connections. Traditional matrix multiplication does not share any parameters



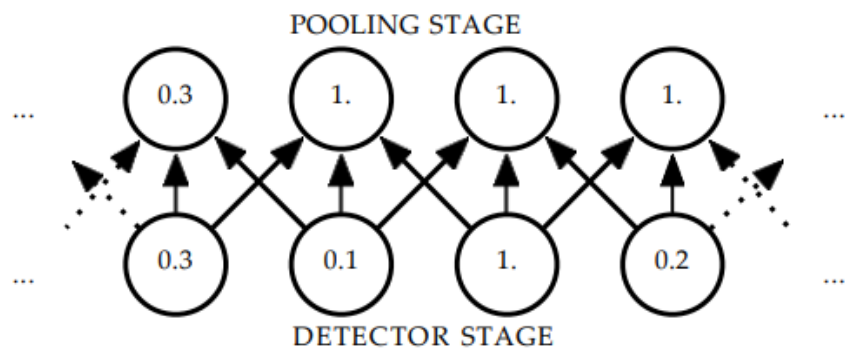
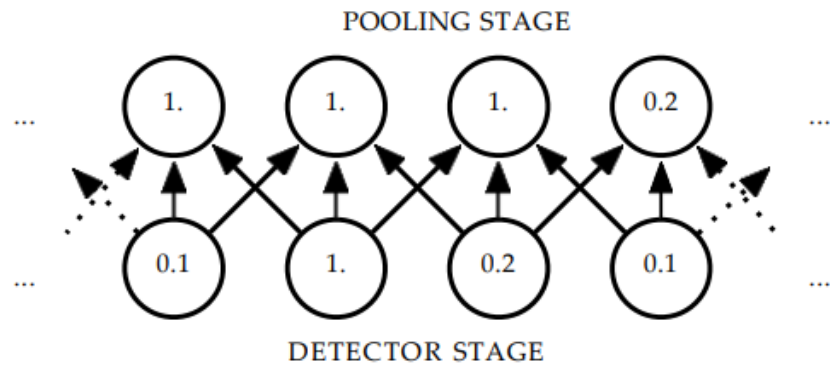
Growing Receptive Fields

Pooling

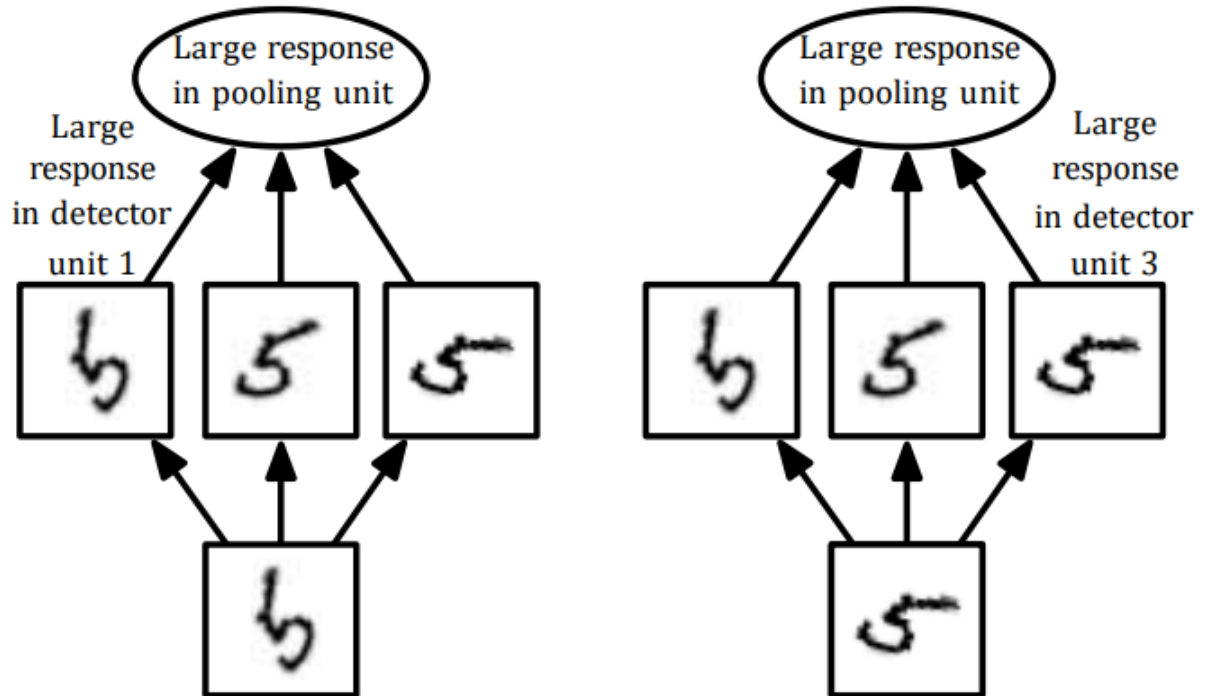
A typical layer of a convolutional network consists of three stages. In the first stage, the layer performs several convolutions in parallel to produce a set of linear activations. In the second stage, each linear activation is run through a nonlinear activation function, such as the rectified linear activation function. This stage is sometimes called the detector stage. In the third stage, we use a pooling function to modify the output of the layer further. A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs. For example, the max-pooling operation reports the maximum output within a rectangular neighborhood. Other popular pooling functions include the average of a rectangular neighborhood which reports the average output within a rectangular neighborhood.



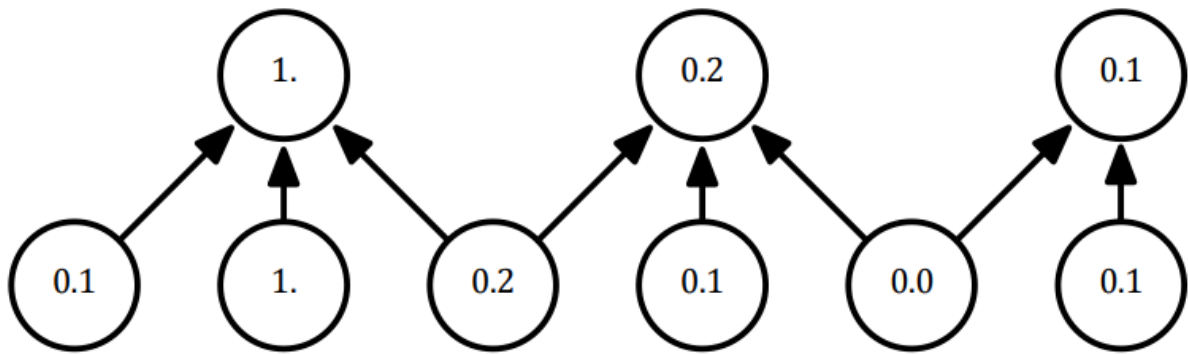
The components of a typical convolutional neural network layer



Max Pooling and Invariance to Translation



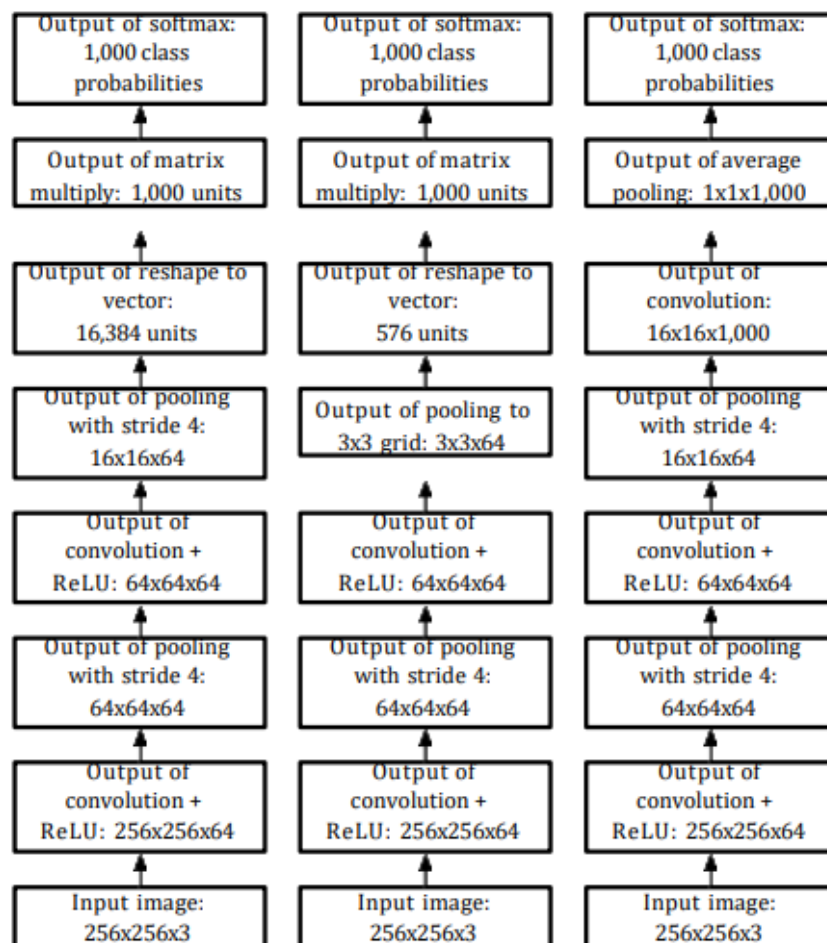
Cross-Channel Pooling and Invariance to Learned Transformations



Pooling with Downsampling

Convolution and Pooling as an Infinitely Strong Prior

Priors can be considered weak or strong depending on how concentrated the probability density in the prior is. A weak prior is a prior distribution with high entropy, such as a Gaussian distribution with high variance. Such a prior allows the data to move the parameters more or less freely. A strong prior has very low entropy, such as a Gaussian distribution with low variance. Such a prior play a more active role in determining where the parameters end up.



Examples of architectures for classification with convolutional networks.

An infinitely strong prior places zero probability on some parameters and say that these parameter values are completely forbidden, regardless of how much support the data give to those values. We can imagine a convolutional net as being similar to a fully connected net, but with an infinitely strong prior over its weights. This infinitely strong prior says that the weights for one hidden unit must be identical to the weights of its neighbor but shifted in space. The prior also says that the weights must be zero, except for in the small, spatially contiguous receptive field assigned to that hidden unit. Overall, we can think of the use of convolution as introducing an infinitely strong prior probability distribution over the parameters of a layer. This prior says that the function the layer should learn contains only local interactions and is equivariant to translation. Likewise, the use of pooling is infinitely strong prior that each unit should be invariant to small translations.

Variants of the Basic Convolution Function

First, when we refer to convolution in the context of neural networks, we usually actually mean an operation that consists of many applications of convolution in parallel. This is because convolution with a single kernel can extract only one kind of feature, albeit at many spatial locations. Usually we want each layer of our network to extract many kinds of features, at many locations.

Additionally, the input is usually not just a grid of real values. Rather, it is a grid of vector-valued observations.

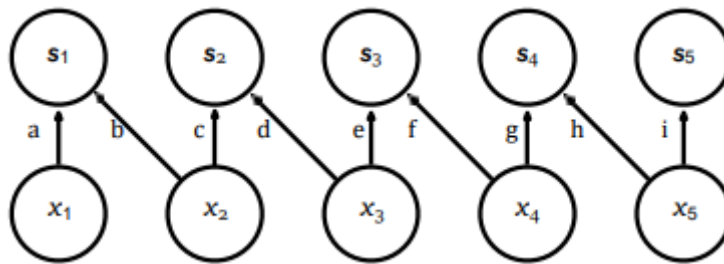
Because convolutional networks usually use multichannel convolution, the linear operations they are based on are not guaranteed to be commutative, even if kernel flipping is used. These multichannel operations are only commutative if each operation has the same number of output channels as input channels.

We may want to skip over some positions of the kernel to reduce the computational cost. We can think of this as downsampling the output of the full convolution function. If we want to sample only every s pixels in each direction in the output, then we can define a downsampled convolution function c such that

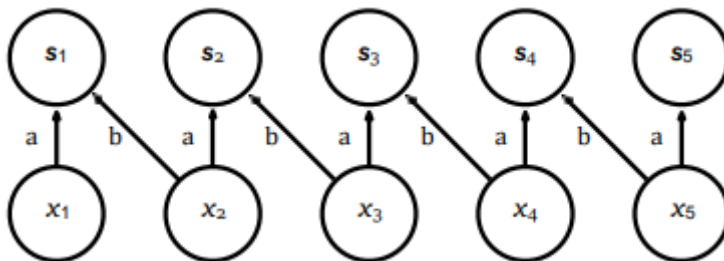
$$Z_{i,j,k} = c(\mathbf{K}, \mathbf{V}, s)_{i,j,k} = \sum_{l,m,n} [V_{l,(j-1) \times s + m, (k-1) \times s + n} K_{i,l,m,n}]$$

We refer to s as the stride of this downsampled convolution. It is also possible to define a separate stride for each direction of motion.

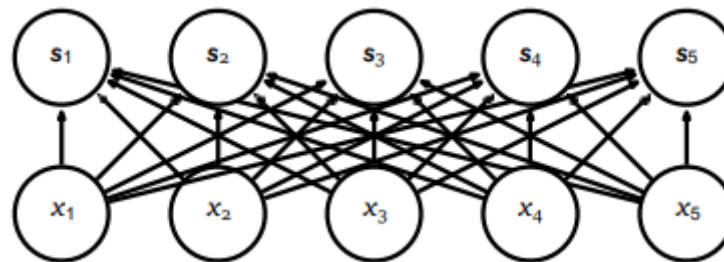
Kinds of connectivity



Local connection: like convolution, but no sharing

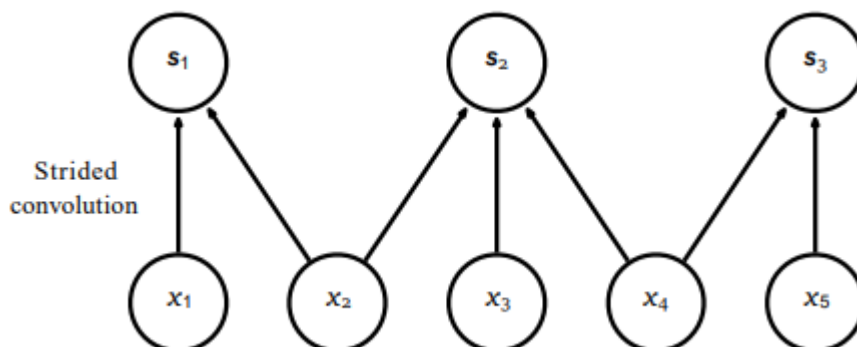


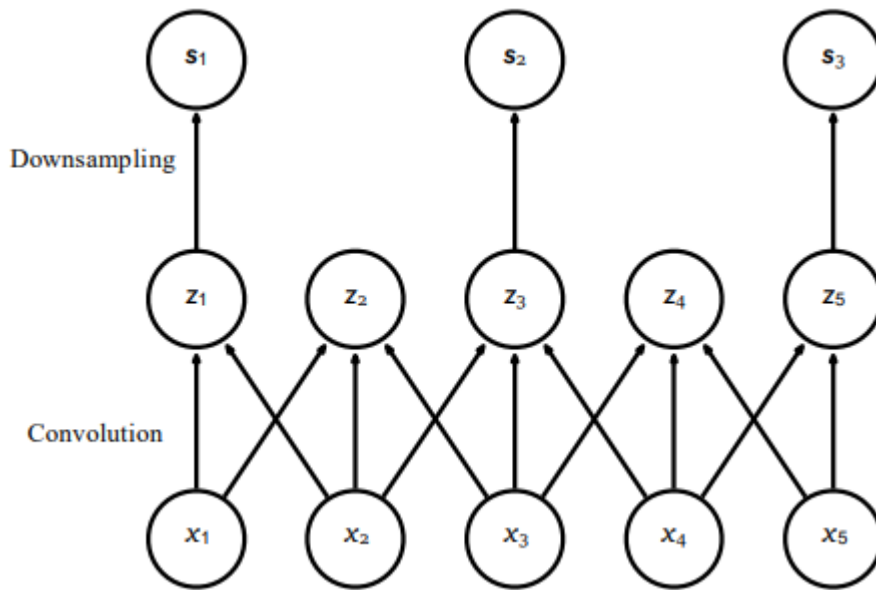
Convolution



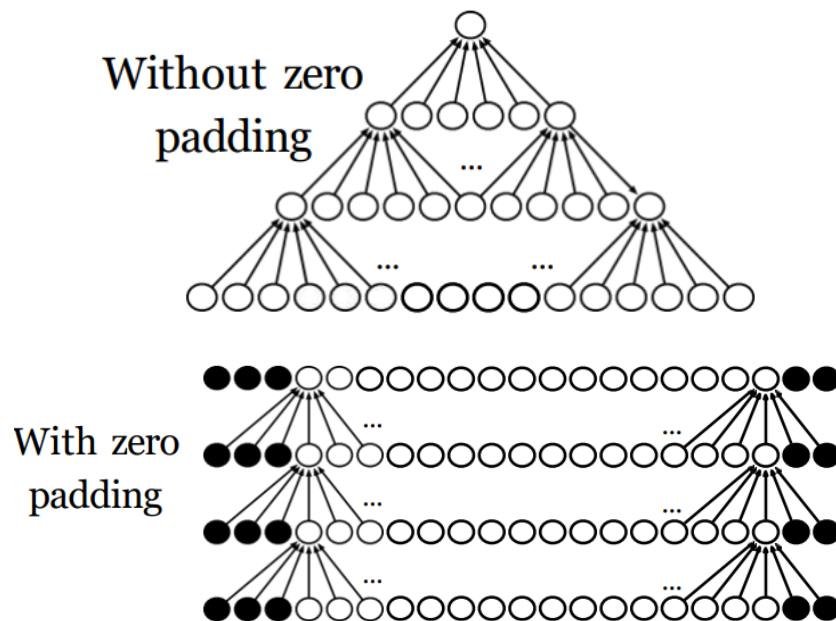
Fully connected

Convolution with Stride



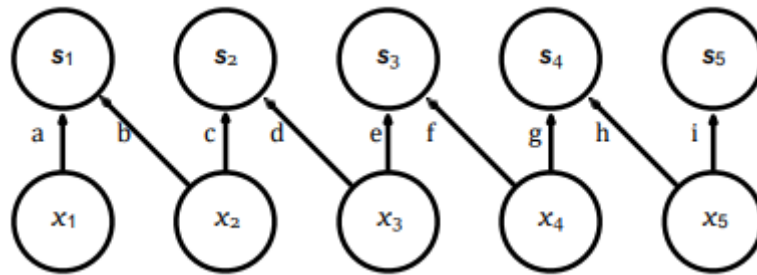


Zero Padding Controls Size

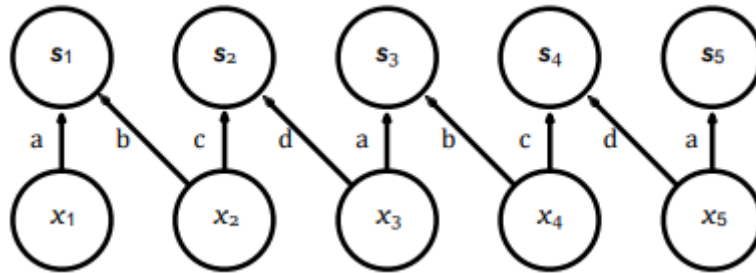


Tiled convolution

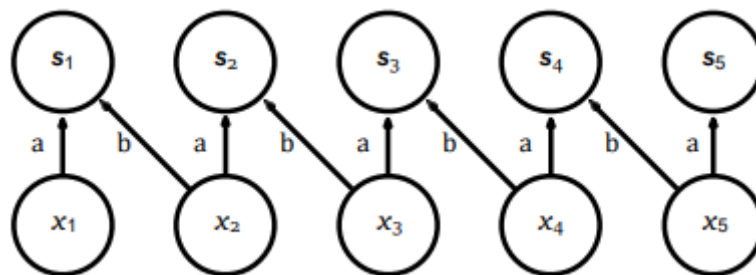
It offers a compromise between a convolutional layer and a locally connected layer. Rather than learning a separate set of weights at every spatial location, we learn a set of kernels that we rotate through as we move through space. This means that immediately neighboring locations will have different filters, as in a locally connected layer, but the memory requirements for storing the parameters will increase only by a factor of the size of this set of kernels, rather than by the size of the entire output feature map.



Local connection (no sharing)



Tiled convolution (cycle between groups of shared parameters)

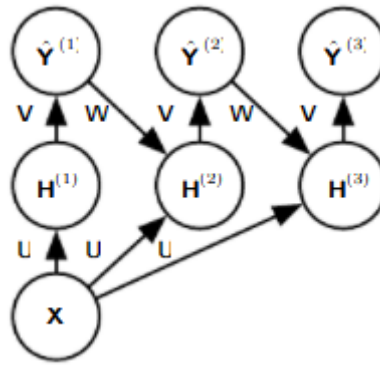


Convolution (one group shared everywhere)

Structured Outputs

Convolutional networks can be used to output a high-dimensional structured object, rather than just predicting a class label for a classification task or a real value for a regression task. Typically this object is just a tensor, emitted by a standard convolutional layer. For example, the model might emit a tensor S , where $S_{i,j,k}$ is the probability that pixel (j, k) of the input to the network belongs to class i . This allows the model to label every pixel in an image and draw precise masks that follow the outlines of individual objects.

One strategy for pixel-wise labeling of images is to produce an initial guess of the image labels, then refine this initial guess using the interactions between neighboring pixels. Repeating this refinement step several times corresponds to using the same convolutions at each stage, sharing weights between the last layers of the deep net. This makes the sequence of computations performed by the successive convolutional layers with weights shared across layers a particular kind of recurrent network.



An example of a recurrent convolutional network for pixel labeling

Once a prediction for each pixel is made, various methods can be used to further process these predictions to obtain a segmentation of the image into regions. The general idea is to assume that large groups of contiguous pixels tend to be associated with the same label. Graphical models can describe the probabilistic relationships between neighboring pixels.

Data Types

The data used with a convolutional network usually consist of several channels, each channel being the observation of a different quantity at some point in space or time.

One advantage to convolutional networks is that they can also process inputs with varying spatial extents. These kinds of input simply cannot be represented by traditional, matrix multiplication-based neural networks. This provides a compelling reason to use convolutional networks even when computational cost and overfitting are not significant issues.

For example, consider a collection of images in which each image has a different width and height. It is unclear how to model such inputs with a weight matrix of fixed size. Convolution is straightforward to apply; the kernel is simply applied a different number of times depending on the size of the input, and the output of the convolution operation scales accordingly. Convolution may be viewed as matrix multiplication; the same convolution kernel induces a different size of doubly block-circulant matrix for each size of input. Sometimes the output of the network as well as the input is allowed to have variable size, for example, if we want to assign a class label to each pixel of the input. In this case, no further design work is necessary. In other cases, the network must produce some fixed-size output, for example, if we want to assign a single class label to the entire image. In this case, we must make some additional design steps, like inserting a pooling layer whose pooling regions scale in size proportional to the size of the input, to maintain a fixed number of pooled outputs.